AnyAccess

P2P Remote Desktop for Windows

One PC shares its screen; the other views and controls it — mouse, keyboard, chat, and file transfer — over a single direct TCP connection. No account, no server in the middle, no internet required (LAN works out of the box).

Requirement	Detail
OS	Windows 10 / 11
Python	3.10+ (tested on 3.14)
Packages	pip install pillow mss
GUI	tkinter (ships with Python)

Contents: 1. The two apps · 2. Start the Host · 3. Start Sharing · 4. Connect the Viewer · 5. Using the session · 6. How it works · 7. Security · 8. Testing

1. The Two Apps

AnyAccess is two programs that talk to each other over TCP:

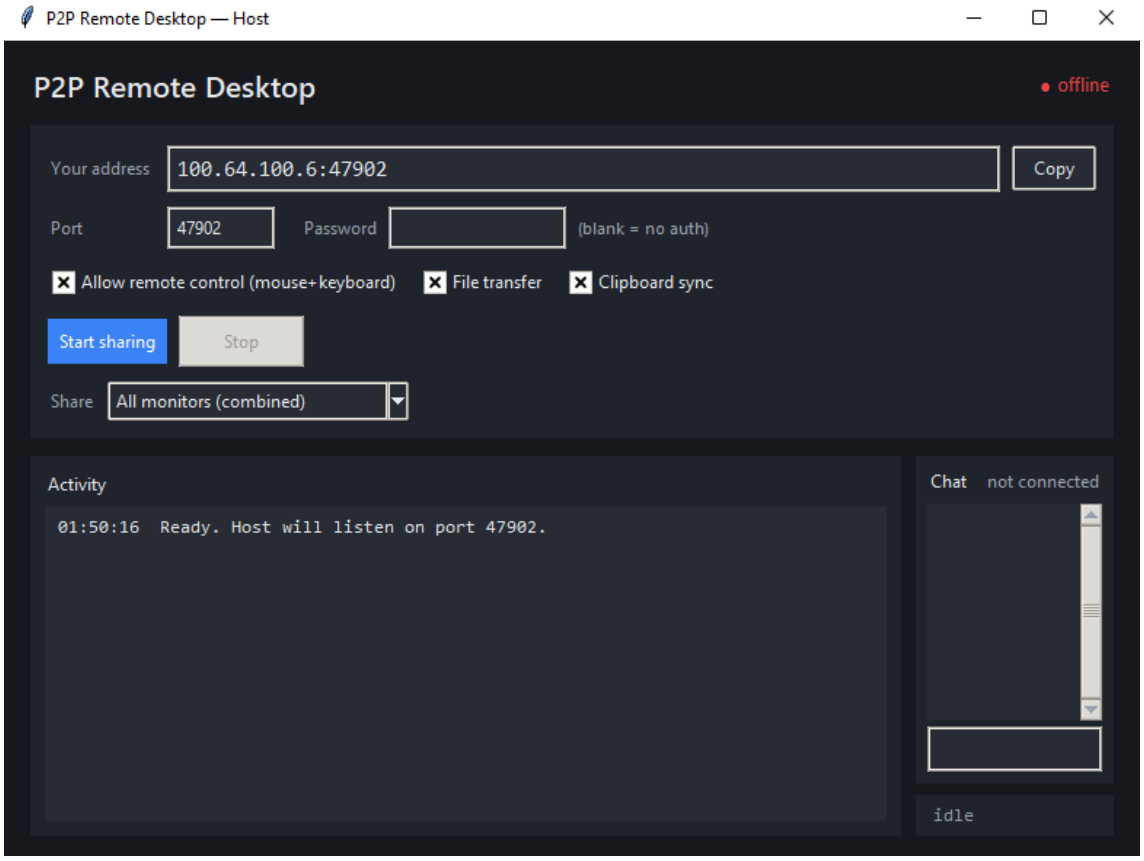
App	Runs on	What it does
host.py	The PC you want to CONTROL	Captures the screen, injects your mouse/keyboard
client.py	The PC you control FROM	Shows the remote screen, forwards your input

Both must be on the same network (LAN) — or reachable over the internet with a port forward / VPN.

2. Start the Host

On the PC you want to control, double-click **start-host.bat** (or run `python host.py`).

The window shows your address, port, password field, permission checkboxes, and a monitor picker. Set a password (recommended), choose what to allow, then click **Start sharing**.

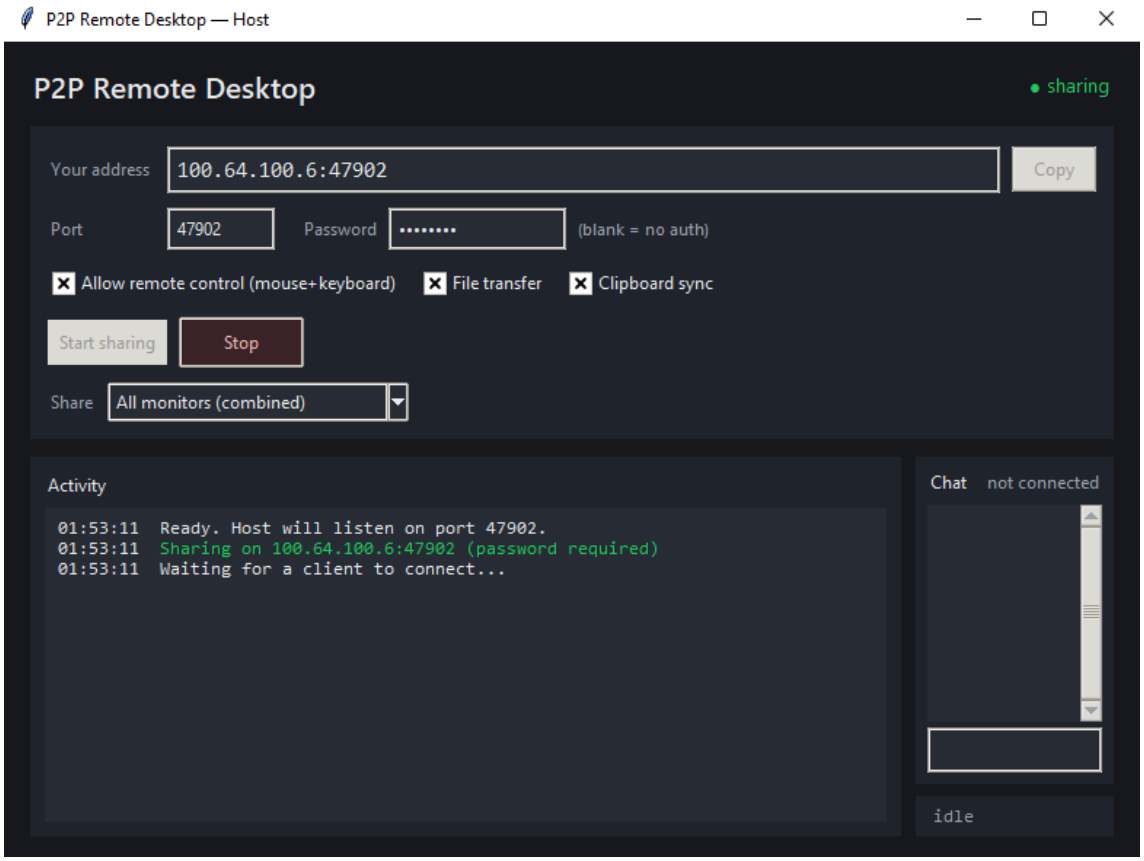


Step 1 — Host window, offline and ready to share

> If Windows asks to allow Python through the firewall, click **Allow** on private networks.

3. Start Sharing

Click the blue **Start sharing** button. The badge top-right turns green (● sharing) and the Activity log confirms the listening address. Click **Copy** to put it on your clipboard.



Step 2 — Host is sharing; green badge, address visible

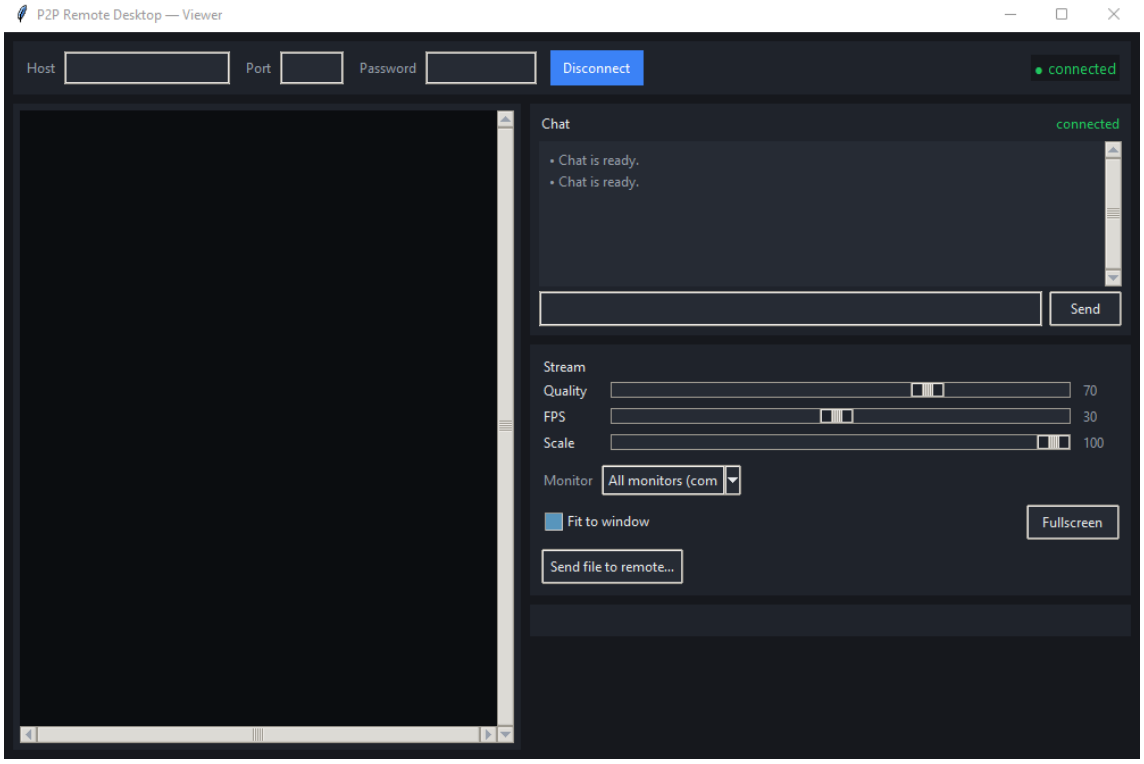
4. Connect from the Viewer

On the other PC, double-click **start-client.bat** (or run `python client.py`).

Then:

- 1. Paste the host's address into **Host**
- 2. Enter the same **Port** (default 47800)
- 3. Type the **Password** you set on the host
- 4. Click **Connect**

The badge turns green (● connected) and the remote screen appears in the canvas. Your mouse and keyboard now control the remote PC.



Step 3 — Viewer connected; remote screen streaming in the canvas

5. Using the Session

Once connected, everything works live:

Feature	How to use it
Remote control	Move the mouse over the canvas; click, drag, right-click, scroll. Type on your keyboard — it goes to the re
Chat	Type in the chat box (bottom right) and press Enter. Messages appear on both sides.
File transfer	Click 'Send file to remote…' and pick a file. The host chooses where to save it.
Stream quality	Drag the Quality / FPS / Scale sliders (right panel) to trade sharpness for speed. Applies instantly.
Fullscreen	Press F11 or click Fullscreen. Press Esc to exit.
Clipboard	Copy text on one side; it appears on the other's clipboard.

Viewer shortcuts

Key	Action
F11	Toggle fullscreen
Esc	Leave fullscreen
F5	Reconnect
Ctrl+L	Connect / disconnect

Stopping: click **Disconnect** on the viewer, or **Stop** on the host. Only one viewer can connect at a time; a second connection is refused while a session is live.

6. How It Works

Everything runs on **one TCP connection** with a simple length-prefixed frame:

```
[type: u32] [length: u32] [payload bytes]
```

Screen frames, input events, chat, file chunks, and clipboard sync all interleave safely on the same socket because every message is self-framed.

Data flow

- host.py captures the screen with mss at your target FPS, encodes JPEG (quality/scale sliders), and streams frames. Incoming events are queued and drained on the send loop, so a slow client never blocks the host's GUI.
- client.py runs a reader thread that demultiplexes the stream; JPEG frames go through a small bounded queue (old frames are dropped, keeping the view live), everything else goes to the Tk UI.
- wininput.py injects mouse/keyboard through Win32 SendInput with virtual-desktop absolute coordinates — correct on any monitor layout and DPI.

Design choices that keep it smooth

- **Backpressure by dropping:** if the link is slow, old frames are discarded so the view stays live instead of lagging behind.
- **Rate limiting:** mouse moves are capped at ~60 Hz so the wire never floods.
- **Keepalive:** PING every 2 s; the viewer shows round-trip latency in its status bar.

7. Security — Read This

v1 authenticates with a **plaintext password over an unencrypted TCP connection**. That is fine on a trusted LAN, but **do not expose it to the internet** without a VPN or SSH tunnel: the password and all traffic can be sniffed, and anyone who connects can control the PC.

Roadmap (not in v1): TLS on the connection, AnyDesk-style ID + rendezvous server for NAT traversal, per-session access tokens.

8. Testing

Three automated suites, all green (34/34 checks):

Suite	Checks	What it verifies
smoke_test.py	14	Protocol: framing, auth, frames, input injection, chat, 300 KB file integrity
gui_drive_test.py	10	Real GUI widgets: buttons, canvas clicks, chat both ways
live_session_test.py	10	Live session: real screen capture, streaming, file transfer

> Run them with `run-tests.bat` or `python smoke_test.py`.